

Scrum: Structured Sprints

Overview

Scrum is an Agile framework that organizes development into short, fixed-length iterations called **sprints**, typically lasting **2–4 weeks**. Each sprint begins with a **planning session** and ends with a **review and retrospective**.

While the scope of work can be reprioritized at the start of each sprint, once a sprint begins, no changes are allowed. This balance of **stability within sprints** and **flexibility between them** ensures both predictability and adaptability.

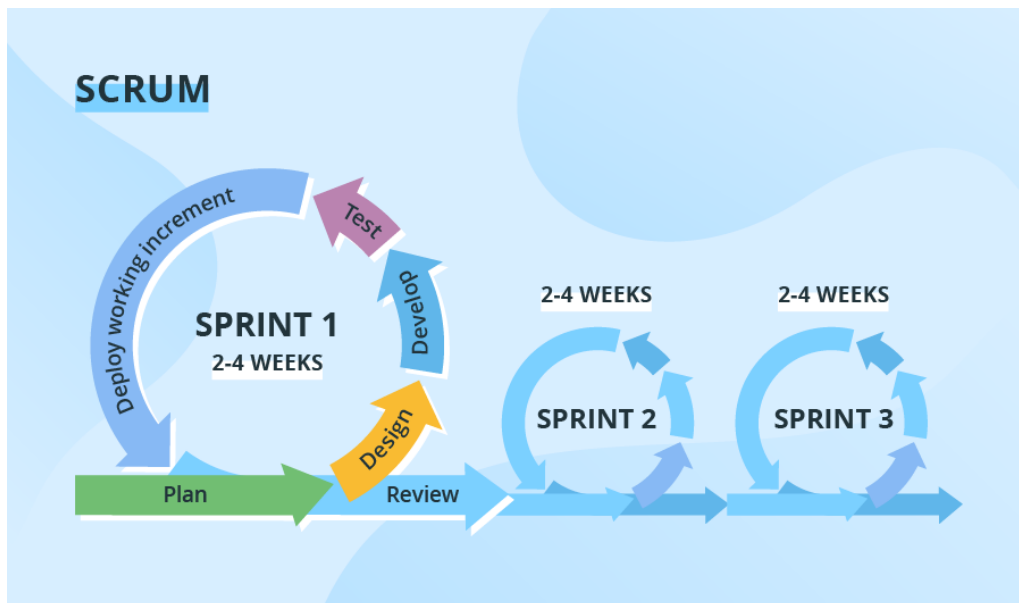


Figure 1: Scrum Sprint Cycle: Plan → Develop → Test → Review → Retrospect

Core Principles

Scrum emphasizes well-defined roles, regular ceremonies, and iterative improvement:

- **Roles:** Product Owner, Scrum Master, and Development Team.
- **Ceremonies:** Sprint Planning, Daily Stand-ups, Sprint Review, and Sprint Retrospective.
- **Artifacts:** Product Backlog, Sprint Backlog, and Increment (working software).

Team Composition

Scrum teams are intentionally small, typically **5–9 members**, allowing for close collaboration and rapid feedback. A balanced Scrum team often includes:

- 3 Developers

- 1 QA Engineer
- 1 Product Owner / Project Manager
- 1 Scrum Master

This compact structure minimizes duplication, fosters accountability, and ensures timely updates.

Adoption and Popularity

Scrum is by far the most widely adopted Agile framework:

- **70%** of Agile teams use Scrum (*StarAgile Report*).
- A **2024 Global Agile Survey** reports **63%** adoption.
- At ScienceSoft, **80%** of Agile projects use Scrum or ScrumBut.

Benefits vs. Challenges

Benefits	Challenges
Provides a clear, structured process while adapting to change.	Requires teams to fully embrace Agile principles to be effective.
Maintains focus on priority work through sprint goals.	Becomes time-consuming if ceremonies lose focus.
Ensures transparent progress and regular feedback loops.	Requires self-discipline to sustain code quality.
Encourages accountability and team ownership.	Doesn't prescribe specific engineering practices like TDD.
Facilitates faster delivery and higher-quality releases.	May be inefficient for highly volatile or non-iterative work.

Best For

- **Feature-driven projects** with a clear vision but evolving requirements.
- Projects requiring **regular validation** through demos and user feedback.
- **Customer-facing apps**, SaaS modules, and enterprise software requiring iteration-based development.

Not Ideal For

- **Fast-changing projects** with daily priority shifts.
- Work that doesn't produce visible results each sprint (e.g., database refactoring, middleware updates).
- Teams lacking Agile experience or unable to meet daily collaboration needs.

Key Insight

Scrum combines *discipline and flexibility*, ensuring that teams deliver value predictably while continuously adapting to feedback and evolving requirements.